STRUCTURED PROGRAMMING IN C

COMPLETE TVET / KNEC STUDY NOTES

TABLE OF CONTENTS

1. Introduction to Structured Programming and C
2. Algorithms and Flowcharts
3. Structure of a C Program
4. Variables, Constants and Data Types
5. Operators and Expressions
6. Input and Output
7. Decision-Making Statements
8. Looping Statements
9. Functions
10. Arrays
11. Strings
12. Pointers
13. Structures, Unions and Enumerations
14. File Handling, Practical Projects and Revision

TOPIC 1: INTRODUCTION TO STRUCTURED PROGRAMMING AND C

1.1 What is Programming?

Programming is the process of writing instructions that tell a computer how to perform a particular task.

A program is a collection of instructions written using a programming language.

Examples of programming languages:

- C
- C++
- Python
- Java

- JavaScript
- C#
- PHP

1.2 What is Structured Programming?

Structured programming is a programming approach that divides a program into smaller, logical sections or procedures.

It emphasizes:

- Sequence
- Selection
- Iteration
- Functions/procedures
- Modular programming
- Clear program structure

The three fundamental control structures are:

1. Sequence

Instructions execute from top to bottom.

```
#include <stdio.h>

int main() {
    int a = 10;
    int b = 20;
    int sum = a + b;

    printf("Sum = %d", sum);

    return 0;
}
```

Execution:

```
a = 10
↓
b = 20
↓
sum = a + b
↓
display result
```

2. Selection

The program makes a decision.

Examples:

```
if
if...else
else if
switch
```

3. Iteration

A section of code is repeated.

Examples:

```
for
while
do...while
```

1.3 What is C?

C is a general-purpose, procedural programming language developed by Dennis Ritchie at Bell Labs in the early 1970s.

C is widely used for:

- Operating systems
- Embedded systems
- Device drivers
- Compilers
- System software
- Networking software
- Microcontrollers
- General programming education

1.4 Characteristics of C

C has the following characteristics:

1. Simple
2. Fast
3. Portable

4. Structured
5. Procedural
6. Case-sensitive
7. Supports pointers
8. Supports functions
9. Provides low-level memory access
10. Suitable for system programming

1.5 Advantages of C

- Fast execution
- Portable
- Efficient memory usage
- Supports structured programming
- Large number of libraries
- Suitable for learning programming concepts
- Allows direct memory manipulation

1.6 Disadvantages of C

- No automatic garbage collection
- Pointers can cause memory problems
- Less built-in protection than some modern languages
- Manual memory management can be difficult
- Does not provide traditional object-oriented programming features

---

1.7 Compiler

A compiler translates source code written by a programmer into machine code that the computer can execute.

Example:

```
C Source Code
      ↓
   Compiler
      ↓
Machine Code
```

```
    ↓
Program
```

Common C compilers include:

- GCC
- Clang
- Microsoft Visual C
- MinGW

1.8 Source Code

Source code is the human-readable code written by a programmer.

Example:

```
#include <stdio.h>

int main() {
    printf("Hello World");
    return 0;
}
```

TOPIC 2: ALGORITHMS AND FLOWCHARTS

2.1 Algorithm

An algorithm is a step-by-step procedure used to solve a problem.

Example: Algorithm for calculating the area of a rectangle.

```
START
 ↓
Input length
 ↓
Input width
 ↓
Calculate area = length × width
 ↓
Display area
 ↓
END
```

2.2 Characteristics of a Good Algorithm

A good algorithm should have:

1. Clearly defined steps
2. A definite beginning
3. A definite ending
4. Correct results
5. Finite number of steps
6. Efficient operations

2.3 Pseudocode

Pseudocode is an informal way of describing an algorithm using simple language.

Example:

```
START

INPUT length
INPUT width

area = length * width

DISPLAY area

END
```

2.4 Flowchart

A flowchart is a graphical representation of an algorithm.

Common flowchart symbols:

Symbol	Meaning
Oval	Start/End
Rectangle	Process
Parallelogram	Input/Output
Diamond	Decision

Symbol	Meaning
Arrow	Flow direction

2.5 Example: Even or Odd Number

Algorithm:

```
START
INPUT number
IF number MOD 2 == 0
    DISPLAY "Even"
ELSE
    DISPLAY "Odd"
END IF
END
```

C implementation:

```
#include <stdio.h>

int main() {
    int number;

    printf("Enter a number: ");
    scanf("%d", &number);

    if (number % 2 == 0) {
        printf("Even number");
    } else {
        printf("Odd number");
    }

    return 0;
}
```

TOPIC 3: STRUCTURE OF A C PROGRAM

3.1 Basic C Program

```
#include <stdio.h>

int main() {

    printf("Hello World");
```

```
    return 0;
}
```

3.2 Explanation

`#include <stdio.h>`

Includes the standard input/output library.

It provides functions such as:

```
printf()
scanf()
```

`main()`

The main function is where program execution begins.

`{ }`

Curly braces define a block of code.

`printf()`

Displays information on the screen.

`return 0`

Indicates that the program finished successfully.

3.3 Comments

Comments are notes written inside source code.

Single-line comment

```
// This is a comment
```

Multi-line comment

```
/*
This is a
multi-line comment
*/
```

3.4 C Syntax

C statements normally end with a semicolon.

```
int age = 20;
printf("%d", age);
```

Missing semicolons can cause compilation errors.

3.5 Case Sensitivity

C is case-sensitive.

These are different:

```
age
Age
AGE
```

Similarly:

```
printf()
```

is correct, while:

```
Printf()
```

is different.

3.6 Escape Sequences

Common escape sequences:

Escape	Meaning
`\n`	New line
`\t`	Tab
`\"`	Double quotation

Escape	Meaning
`\\`	Backslash
`\b`	Backspace

Example:

```
printf("Name:\tJose\n");
printf("Course:\tICT\n");
```

Output:

```
Name:   Jose
Course: ICT
```

TOPIC 4: VARIABLES, CONSTANTS AND DATA TYPES

4.1 Variable

A variable is a named memory location used to store data.

Example:

```
int age = 20;
```

Here:

- `int` = data type
- `age` = variable
- `20` = value

4.2 Declaring Variables

```
int age;
float salary;
char grade;
```

4.3 Initializing Variables

```
int age = 20;
float salary = 35000.50;
char grade = 'A';
```

4.4 Common C Data Types

Data Type	Purpose	Example
`int`	Whole numbers	`20`
`float`	Decimal numbers	`12.5`
`double`	More precise decimals	`12.5678`
`char`	Single character	`'A'`

4.5 Integer

```
int age = 25;
```

4.6 Float

```
float price = 99.50;
```

4.7 Double

```
double distance = 12345.6789;
```

4.8 Character

```
char grade = 'A';
```

A character uses single quotes.

4.9 Constants

A constant is a value that should not change during program execution.

Using `const`:

```
const float PI = 3.14159;
```

Using `#define`:

```
#define PI 3.14159
```

4.10 Variable Naming Rules

A variable name:

- Cannot start with a number
- Cannot contain spaces
- Can contain letters, numbers and underscores
- Cannot be a C keyword
- Is case-sensitive

Correct:

```
studentName
student_name
mark1
age
```

Incorrect:

```
student name
1student
student-name
```

TOPIC 5: OPERATORS AND EXPRESSIONS

5.1 Arithmetic Operators

Operator	Meaning
`+`	Addition
`-`	Subtraction
`*`	Multiplication
`/`	Division
`%`	Modulus

Example:

```
int a = 20;
int b = 6;

printf("%d\n", a + b);
printf("%d\n", a - b);
printf("%d\n", a * b);
printf("%d\n", a / b);
printf("%d\n", a % b);
```

5.2 Relational Operators

Used to compare values.

Operator	Meaning
==	Equal
!=	Not equal
>	Greater than
<	Less than
>=	Greater/equal
<=	Less/equal

Example:

```
if (age >= 18) {
    printf("Adult");
}
```

5.3 Logical Operators

Operator	Meaning
&&	AND
`	
!	NOT

Example:

```
if (age >= 18 && citizen == 1) {
    printf("Eligible");
}
```

5.4 Assignment Operators

```
=
+=
-=
*=
/=
%=
```

Example:

```
int x = 10;

x += 5;
```

Equivalent to:

```
x = x + 5;
```

5.5 Increment and Decrement

```
x++;
x--;
```

Example:

```
int x = 5;

x++;

printf("%d", x);
```

Output:

```
6
```

5.6 Operator Precedence

Example:

```
int result = 10 + 5 * 2;
```

Multiplication happens first.

Result:

```
20
```

Using parentheses:

```
int result = (10 + 5) * 2;
```

Result:

```
30
```

TOPIC 6: INPUT AND OUTPUT

6.1 Output Using printf()

```
printf("Hello");
```

6.2 Format Specifiers

Specifier	Data
`%d`	int
`%f`	float
`%lf`	double with `scanf`
`%c`	char
`%s`	string

Example:

```
int age = 20;
printf("Age = %d", age);
```

6.3 Input Using scanf()

```
int age;

printf("Enter age: ");
scanf("%d", &age);
```

The `&` provides the address where `scanf()` should store the input.

6.4 Multiple Inputs

```
int age;
float height;

printf("Enter age: ");
scanf("%d", &age);

printf("Enter height: ");
scanf("%f", &height);
```

6.5 Character Input

```
char grade;

printf("Enter grade: ");
scanf(" %c", &grade);
```

The space before `%c` helps consume leftover whitespace.

6.6 String Input

```
char name[50];

printf("Enter name: ");
scanf("%49s", name);
```

Note that an array name already represents an address, so `&` is normally not used here.

For names containing spaces, `fgets()` is preferable:

```
fgets(name, sizeof(name), stdin);
```

TOPIC 7: DECISION-MAKING STATEMENTS

Decision-making allows a program to choose between different actions.

7.1 if Statement

Syntax:

```
if (condition) {
    statements;
}
```

Example:

```
int age = 20;

if (age >= 18) {
    printf("Adult");
}
```

7.2 if...else

```
int age;

printf("Enter age: ");
scanf("%d", &age);

if (age >= 18) {
    printf("Adult");
} else {
    printf("Minor");
}
```

7.3 else if

```
int mark;

printf("Enter mark: ");
scanf("%d", &mark);

if (mark >= 80) {
    printf("Grade A");
} else if (mark >= 70) {
    printf("Grade B");
} else if (mark >= 60) {
    printf("Grade C");
} else if (mark >= 50) {
    printf("Grade D");
} else {
    printf("Grade F");
}
```

7.4 Nested if

An `if` statement inside another `if` statement is called nested if.

```
if (age >= 18) {

    if (hasID == 1) {
        printf("Allowed");
    }
}
```

7.5 switch Statement

The `switch` statement is useful when selecting from several fixed options.

Example:

```
#include <stdio.h>

int main() {
    int choice;

    printf("1. Deposit\n");
    printf("2. Withdraw\n");
    printf("3. Check Balance\n");

    printf("Enter choice: ");
    scanf("%d", &choice);

    switch (choice) {
        case 1:
            printf("Deposit selected");
            break;

        case 2:
            printf("Withdraw selected");
            break;

        case 3:
            printf("Balance selected");
            break;

        default:
            printf("Invalid choice");
    }

    return 0;
}
```

TOPIC 8: LOOPING STATEMENTS

Loops repeat a block of code.

C has three major loops:

1. `for`
2. `while`
3. `do...while`

8.1 for Loop

Syntax:

```
for (initialization; condition; update) {
    statements;
}
```

Example:

```
for (int i = 1; i <= 5; i++) {
    printf("%d\n", i);
}
```

Output:

```
1
2
3
4
5
```

8.2 while Loop

```
int i = 1;

while (i <= 5) {
    printf("%d\n", i);
    i++;
}
```

8.3 do...while Loop

A `do...while` loop executes at least once.

```
int i = 1;

do {
    printf("%d\n", i);
    i++;
} while (i <= 5);
```

8.4 break

Stops a loop.

```
for (int i = 1; i <= 10; i++) {

    if (i == 5) {
        break;
    }

    printf("%d\n", i);
}
```

8.5 continue

Skips the current iteration.

```
for (int i = 1; i <= 5; i++) {

    if (i == 3) {
        continue;
    }

    printf("%d\n", i);
}
```

Output:

```
1
2
4
5
```

8.6 Nested Loops

A loop inside another loop.

Example:

```
for (int i = 1; i <= 3; i++) {

    for (int j = 1; j <= 3; j++) {
        printf("%d %d\n", i, j);
    }
}
```

8.7 Multiplication Table

```
#include <stdio.h>

int main() {
    int number;

    printf("Enter number: ");
    scanf("%d", &number);

    for (int i = 1; i <= 12; i++) {
        printf("%d x %d = %d\n",
               number, i, number * i);
    }

    return 0;
}
```

TOPIC 9: FUNCTIONS

9.1 What is a Function?

A function is a block of code designed to perform a specific task.

Functions make programs:

- Modular
- Reusable
- Easier to test
- Easier to maintain
- Easier to understand

9.2 Function Structure

```
return_type function_name(parameters) {
    statements;
}
```

9.3 Function Without Parameters

```
#include <stdio.h>

void greeting() {
    printf("Welcome to C programming");
}

int main() {

    greeting();

    return 0;
}
```

9.4 Function With Parameters

```
void greet(char name[]) {
    printf("Hello %s", name);
}
```

Calling it:

```
greet("Jose");
```

9.5 Function Returning a Value

```
int add(int a, int b) {
    return a + b;
}
```

Use:

```
int result = add(10, 20);

printf("%d", result);
```

9.6 Function Prototype

A function prototype tells the compiler about a function before it is used.

```
int add(int, int);
```

Example:

```
#include <stdio.h>

int add(int a, int b);

int main() {

    int result = add(10, 20);

    printf("Result = %d", result);

    return 0;
}

int add(int a, int b) {
    return a + b;
}
```

9.7 Call by Value

A copy of the value is passed to a function.

```
void change(int x) {
    x = 100;
}
```

Changing `x` does not change the original variable.

9.8 Passing Address to a Function

Pointers can be used when a function needs to modify the original variable.

```
void change(int *x) {
    *x = 100;
}
```

Example:

```
int number = 10;

change(&number);
```

Now `number` becomes `100`.

9.9 Recursive Function

A recursive function calls itself.

Example:

```
int factorial(int n) {

    if (n == 0) {
        return 1;
    }

    return n * factorial(n - 1);
}
```

TOPIC 10: ARRAYS

10.1 What is an Array?

An array is a collection of elements of the same data type stored under one variable name.

Example:

```
int marks[5];
```

10.2 Array Initialization

```
int marks[5] = {70, 80, 65, 90, 75};
```

Indexes start from zero.

```
Index:  0   1   2   3   4
Value: 70  80  65  90  75
```

10.3 Accessing an Array

```
printf("%d", marks[0]);
```

Output:

```
70
```

10.4 Loop Through an Array

```
int marks[5] = {70, 80, 65, 90, 75};

for (int i = 0; i < 5; i++) {
    printf("%d\n", marks[i]);
}
```

10.5 Input Array Values

```
int marks[5];

for (int i = 0; i < 5; i++) {
    printf("Enter mark %d: ", i + 1);
    scanf("%d", &marks[i]);
}
```

10.6 Calculate Total

```
int marks[5] = {70, 80, 65, 90, 75};
int total = 0;

for (int i = 0; i < 5; i++) {
    total += marks[i];
}

printf("Total = %d", total);
```

10.7 Calculate Average

```
float average;
```

```
average = total / 5.0;
```

10.8 Two-Dimensional Arrays

A two-dimensional array can represent rows and columns.

```
int matrix[2][3] = {
    {1, 2, 3},
    {4, 5, 6}
};
```

Access:

```
printf("%d", matrix[0][1]);
```

Output:

```
2
```

TOPIC 11: STRINGS

11.1 What is a String?

A string is a sequence of characters terminated by the null character `\0`.

Example:

```
char name[] = "Jose";
```

Internally:

```
J o s e \0
```

11.2 Declaring Strings

```
char name[50];
```

11.3 Initializing a String

```
char name[] = "Jose";
```

11.4 Reading a String

```
char name[50];

printf("Enter name: ");
scanf("%49s", name);
```

For spaces:

```
fgets(name, sizeof(name), stdin);
```

11.5 String Library

Include:

```
#include <string.h>
```

Common functions:

Function	Purpose
`strlen()`	Finds length
`strcpy()`	Copies string
`strcat()`	Joins strings
`strcmp()`	Compares strings

11.6 strlen()

```
char name[] = "Jose";

printf("%zu", strlen(name));
```

Output:

```
4
```

11.7 strcpy()

```
char first[20] = "Jose";
char second[20];

strcpy(second, first);
```

11.8 strcat()

```
char first[50] = "Jose ";
char last[] = "Ndolo";

strcat(first, last);

printf("%s", first);
```

11.9 strcmp()

```
if (strcmp("admin", "admin") == 0) {
    printf("Match");
}
```

TOPIC 12: POINTERS

12.1 What is a Pointer?

A pointer is a variable that stores the memory address of another variable.

Example:

```
int number = 10;
int *ptr = &number;
```

Here:

```
number → stores 10
ptr    → stores address of number
```

12.2 Address Operator &

The `&` operator obtains the memory address of a variable.

```
printf("%p", (void *)&number);
```

12.3 Dereference Operator *

The * operator can access the value stored at an address.

```
printf("%d", *ptr);
```

12.4 Complete Pointer Example

```
#include <stdio.h>

int main() {

    int number = 50;
    int *ptr = &number;

    printf("Value = %d\n", number);
    printf("Address = %p\n", (void *)&number);
    printf("Pointer = %p\n", (void *)ptr);
    printf("Value through pointer = %d\n", *ptr);

    return 0;
}
```

12.5 Pointers and Functions

Pointers allow functions to modify original variables.

```
void change(int *number) {
    *number = 100;
}

int main() {

    int number = 20;

    change(&number);

    printf("%d", number);

    return 0;
}
```

Output:

```
100
```

12.6 Pointers and Arrays

The name of an array is closely related to the address of its first element.

```
int numbers[] = {10, 20, 30};

printf("%d", *(numbers + 1));
```

Output:

```
20
```

12.7 NULL Pointer

A pointer that does not currently point to a valid object can be initialized to `NULL`.

```
int *ptr = NULL;
```

Before dereferencing a pointer, it is good practice to ensure it points to valid storage.

TOPIC 13: STRUCTURES, UNIONS AND ENUMERATIONS

13.1 Structures

A structure allows different types of data to be grouped together.

Example:

```
struct Student {
    int id;
    char name[50];
    float marks;
};
```

13.2 Creating a Structure Variable

```
struct Student student1;
```

13.3 Assigning Values

```
student1.id = 101;
student1.marks = 85.5;
```

For a string:

```
strcpy(student1.name, "Jose");
```

13.4 Complete Structure Example

```
#include <stdio.h>
#include <string.h>

struct Student {
    int id;
    char name[50];
    float marks;
};

int main() {

    struct Student;

    student.id = 101;
    strcpy(student.name, "Jose");
    student.marks = 85.5;

    printf("ID: %d\n", student.id);
    printf("Name: %s\n", student.name);
    printf("Marks: %.2f\n", student.marks);

    return 0;
}
```

13.5 Array of Structures

```
struct Student students[3];
```

Example:

```
for (int i = 0; i < 3; i++) {

    printf("Enter ID: ");
```

```
    scanf("%d", &students[i].id);

    printf("Enter marks: ");
    scanf("%f", &students[i].marks);
}
```

13.6 typedef

`typedef` provides another name for a data type.

Example:

```
typedef struct {
    int id;
    char name[50];
    float marks;
} Student;
```

Now:

```
Student student1;
```

can be used instead of:

```
struct Student student1;
```

13.7 Union

A union is similar to a structure, but its members share the same memory location.

Example:

```
union Data {
    int number;
    float decimal;
    char letter;
};
```

Only one member should generally be treated as containing the currently stored value at a time.

13.8 Enumeration

An enumeration defines named integer constants.

Example:

```
enum Day {
    MONDAY,
    TUESDAY,
    WEDNESDAY,
    THURSDAY,
    FRIDAY
};
```

Use:

```
enum Day today = WEDNESDAY;
```

TOPIC 14: FILE HANDLING, PRACTICAL PROJECTS AND REVISION

14.1 File Handling

File handling allows programs to store data permanently.

Without files, data stored in normal variables disappears when the program ends.

14.2 Opening a File

Use:

```
FILE *file;
```

Open a file:

```
file = fopen("students.txt", "w");
```

14.3 File Modes

Mode	Purpose
`r`	Read

Mode	Purpose
`w`	Write
`a`	Append
`r+`	Read/write
`w+`	Write/read
`a+`	Append/read

14.4 Writing to a File

```
#include <stdio.h>

int main() {

    FILE *file;

    file = fopen("students.txt", "w");

    if (file == NULL) {
        printf("Unable to open file");
        return 1;
    }

    fprintf(file, "Jose 85\n");
    fprintf(file, "Annan 90\n");

    fclose(file);

    return 0;
}
```

14.5 Reading a File

```
#include <stdio.h>

int main() {

    FILE *file;
    char line[100];

    file = fopen("students.txt", "r");

    if (file == NULL) {
        printf("File not found");
        return 1;
    }

    while (fgets(line, sizeof(line), file) != NULL) {
```

```
        printf("%s", line);
    }

    fclose(file);

    return 0;
}
```

14.6 Appending Data

```
FILE *file = fopen("students.txt", "a");

fprintf(file, "Mercy 78\n");

fclose(file);
```

The `a` mode adds new data to the end of the file.

14.7 Important File Functions

Function	Purpose
`fopen()`	Opens a file
`fclose()`	Closes a file
`fprintf()`	Writes formatted data
`fscanf()`	Reads formatted data
`fgets()`	Reads a line
`fputs()`	Writes a string
`fgetc()`	Reads a character
`fputc()`	Writes a character

PRACTICAL PROJECT 1: SIMPLE CALCULATOR

```
#include <stdio.h>

int main() {

    double num1, num2;
    char operator;
```

```
    printf("Enter first number: ");
    scanf("%lf", &num1);

    printf("Enter operator (+ - * /): ");
    scanf(" %c", &operator);

    printf("Enter second number: ");
    scanf("%lf", &num2);

    switch (operator) {

        case '+':
            printf("Result = %.2f", num1 + num2);
            break;

        case '-':
            printf("Result = %.2f", num1 - num2);
            break;

        case '*':
            printf("Result = %.2f", num1 * num2);
            break;

        case '/':
            if (num2 != 0) {
                printf("Result = %.2f", num1 / num2);
            } else {
                printf("Cannot divide by zero");
            }
            break;

        default:
            printf("Invalid operator");
    }

    return 0;
}
```

PRACTICAL PROJECT 2: STUDENT MARKS CALCULATOR

```
#include <stdio.h>

int main() {

    char name[50];
    float mark1, mark2, mark3;
    float total, average;

    printf("Enter student name: ");
    scanf("%49s", name);
```

```
    printf("Enter mark 1: ");
    scanf("%f", &mark1);

    printf("Enter mark 2: ");
    scanf("%f", &mark2);

    printf("Enter mark 3: ");
    scanf("%f", &mark3);

    total = mark1 + mark2 + mark3;
    average = total / 3;

    printf("\nStudent: %s\n", name);
    printf("Total: %.2f\n", total);
    printf("Average: %.2f\n", average);

    if (average >= 80) {
        printf("Grade: A");
    } else if (average >= 70) {
        printf("Grade: B");
    } else if (average >= 60) {
        printf("Grade: C");
    } else if (average >= 50) {
        printf("Grade: D");
    } else {
        printf("Grade: F");
    }

    return 0;
}
```

PRACTICAL PROJECT 3: ATM MENU

```
#include <stdio.h>

int main() {

    int choice;
    double balance = 50000;
    double amount;

    do {

        printf("\n===== ATM MENU =====\n");
        printf("1. Check Balance\n");
        printf("2. Deposit\n");
        printf("3. Withdraw\n");
        printf("4. Exit\n");

        printf("Enter choice: ");
        scanf("%d", &choice);

        switch (choice) {
```

```
            case 1:
                printf("Balance = %.2f\n", balance);
                break;

            case 2:
                printf("Enter amount: ");
                scanf("%lf", &amount);

                if (amount > 0) {
                    balance += amount;
                    printf("Deposit successful\n");
                } else {
                    printf("Invalid amount\n");
                }
                break;

            case 3:
                printf("Enter amount: ");
                scanf("%lf", &amount);

                if (amount <= 0) {
                    printf("Invalid amount\n");
                } else if (amount > balance) {
                    printf("Insufficient balance\n");
                } else {
                    balance -= amount;
                    printf("Withdrawal successful\n");
                }

                break;

            case 4:
                printf("Thank you for using the ATM\n");
                break;

            default:
                printf("Invalid choice\n");
        }

    } while (choice != 4);

    return 0;
}
```

PRACTICAL PROJECT 4: STUDENT MANAGEMENT SYSTEM

A basic student management system can contain:

```
1. Add Student
2. View Students
3. Search Student
```

```
4. Update Marks
5. Delete Student
6. Exit
```

Important concepts used:

- Structures
- Arrays
- Functions
- Loops
- Selection
- Strings
- File handling

Example structure:

```
typedef struct {
    int id;
    char name[50];
    float marks;
} Student;
```

PRACTICAL PROJECT 5: SIMPLE BANKING SYSTEM

Possible functions:

```
1. Create Account
2. Deposit
3. Withdraw
4. Check Balance
5. Exit
```

Possible structure:

```
typedef struct {
    int accountNumber;
    char name[50];
    double balance;
} Account;
```

PRACTICAL PROJECT 6: CONTACT BOOK

A contact book can store:

```
Name
Phone number
Email
Address
```

Example:

```
typedef struct {
    char name[50];
    char phone[20];
    char email[100];
} Contact;
```

Functions:

```
addContact()
viewContacts()
searchContact()
deleteContact()
```

ERROR TYPES IN C

1. Syntax Errors

Errors caused by incorrect C syntax.

Example:

```
int age = 20
```

Missing:

```
;
```

Correct:

```
int age = 20;
```

---

2. Runtime Errors

Errors that occur while the program is running.

Example:

```
int result = 10 / 0;
```

---

3. Logical Errors

The program runs but produces an incorrect result.

Example:

```
average = total / 2;
```

when there are actually three marks.

The correct calculation would be:

```
average = total / 3;
```

DEBUGGING

Debugging is the process of finding and correcting errors in a program.

Common debugging techniques:

- Read compiler messages
- Check variable values
- Use `printf()`
- Check conditions
- Test small sections
- Use a debugger
- Check loops
- Check array indexes
- Check pointers
- Check file operations

BEST PRACTICES IN C

1. Use meaningful variable names.

Instead of:

```
int x;
```

prefer:

```
int studentAge;
```

2. Indent your code.
3. Use comments where useful.
4. Break large programs into functions.
5. Validate user input.
6. Close files after use.
7. Check whether files opened successfully.
8. Avoid accessing arrays outside their bounds.
9. Initialize variables before using them.
10. Be careful when using pointers.

COMMON C HEADER FILES

Header	Purpose
`<stdio.h>`	Input/output
`<stdlib.h>`	General utilities
`<string.h>`	String operations
`<math.h>`	Mathematical functions
`<ctype.h>`	Character operations
`<time.h>`	Date/time functions

Examples:

```
#include <stdio.h>
#include <string.h>
#include <math.h>
```

COMMON C FUNCTIONS

Input/output

```
printf()
scanf()
fgets()
```

String functions

```
strlen()
```

```
strcpy()
strcat()
strcmp()
```

File functions

```
fopen()
fclose()
fprintf()
fscanf()
fgets()
fputs()
```

Mathematical functions

```
sqrt()
pow()
ceil()
floor()
```

C PROGRAM DEVELOPMENT PROCESS

A typical programming process is:

```
Problem Definition
        ↓
Problem Analysis
        ↓
Algorithm
        ↓
Flowchart/Pseudocode
        ↓
Coding
        ↓
Compilation
        ↓
Testing
        ↓
Debugging
        ↓
Documentation
        ↓
Maintenance
```

REVISION QUESTIONS

SECTION A: MULTIPLE CHOICE

1. Which function is the entry point of a C program?

A. `start()`

B. `main()`

C. `begin()`

D. `run()`

Answer: B

---

2. Which header file is used for `printf()`?

A. `string.h`

B. `math.h`

C. `stdio.h`

D. `stdlib.h`

Answer: C

---

3. Which symbol is used to terminate most C statements?

A. `:`

B. `.`

C. `;`

D. `,`

Answer: C

---

4. Which data type stores a whole number?

A. `float`

B. `char`

C. `int`

D. `double`

Answer: C

---

5. Which operator is used for equality comparison?

A. `=`

B. `==`

C. `!=`

D. `=>`

Answer: B

---

6. Which loop is guaranteed to execute at least once?

A. `for`

B. `while`

C. `do...while`

D. `if`

Answer: C

---

7. Which operator obtains the address of a variable?

A. `*`

B. `&`

C. `%`

D. `#`

Answer: B

---

8. Which operator accesses the value stored at a pointer's address?

A. `&`

B. `#`

C. `*`

D. `%`

Answer: C

---

9. Which function opens a file?

A. `open()`

B. `fileopen()`

C. `fopen()`

D. `openfile()`

Answer: C

---

10. Which keyword is used to define a structure?

A. `class`

B. `struct`

C. `record`

D. `object`

Answer: B

SECTION B: SHORT-ANSWER QUESTIONS

1. Define structured programming.
2. Define an algorithm.
3. What is a flowchart?
4. State four characteristics of C.
5. What is a variable?
6. Differentiate between a variable and a constant.
7. State four C data types.
8. What is an operator?
9. Differentiate between `=` and `==`.
10. Explain the purpose of `printf()`.
11. Explain the purpose of `scanf()`.
12. What is a conditional statement?
13. Differentiate between `while` and `do...while`.
14. What is a function?
15. State four advantages of functions.
16. What is an array?
17. Explain what a pointer is.
18. What is a structure?
19. What is file handling?
20. State three types of programming errors.

SECTION C: PRACTICAL QUESTIONS

Question 1

Write a C program that accepts two numbers and displays:

- Sum
- Difference
- Product
- Quotient

Question 2

Write a C program that determines whether a number is:

- Positive
- Negative
- Zero

Question 3

Write a program that accepts five student marks and calculates:

- Total
- Average
- Highest mark
- Lowest mark

Question 4

Write a program that displays numbers from 1 to 100 using a `for` loop.

Question 5

Write a program that calculates the factorial of a number.

Question 6

Write a program that determines whether a number is prime.

Question 7

Create a menu-driven calculator using `switch`.

Question 8

Create a student structure containing:

```
Student ID
Name
Course
Marks
```

Display the student's information.

Question 9

Write a program that stores five student records using an array of structures.

Question 10

Write a program that stores student information in a text file and reads it back.

PRACTICAL EXAM PROJECT

STUDENT MANAGEMENT SYSTEM

Develop a C program that provides the following menu:

```
================================
       STUDENT MANAGEMENT
================================

1. Add Student
2. View Students
3. Search Student
4. Update Marks
5. Delete Student
6. Exit
```

The program should store:

```
Student ID
Student Name
```

```
Course
Marks
Grade
```

The system should calculate grades using:

```
80 - 100 = A
70 - 79  = B
60 - 69  = C
50 - 59  = D
Below 50 = F
```

Recommended functions:

```
void addStudent();
void viewStudents();
void searchStudent();
void updateStudent();
void deleteStudent();
char calculateGrade(float marks);
```

For a more advanced version, use file handling to permanently save the students.

IMPORTANT EXAM TOPICS TO MASTER

Before taking a C programming examination, make sure you understand:

Programming fundamentals

- Algorithms
- Flowcharts
- Pseudocode
- Structured programming
- Program development

C fundamentals

- C syntax
- Variables
- Constants
- Data types
- Keywords
- Comments
- Format specifiers

Operators

- Arithmetic
- Relational
- Logical
- Assignment
- Increment/decrement

Control structures

- `if`
- `if...else`
- `else if`
- Nested `if`
- `switch`
- `for`
- `while`
- `do...while`

Modular programming

- Functions
- Parameters
- Return values
- Function prototypes
- Recursion

Data structures in C

- Arrays
- Strings
- Structures
- Unions
- Enumerations

Memory

- Addresses
- Pointers
- Dereferencing
- Pointers and functions
- Pointers and arrays

File handling

- `fopen()`

- `fclose()`
- `fprintf()`
- `fscanf()`
- `fgets()`
- `fputs()`
- File modes

Debugging

- Syntax errors
- Runtime errors
- Logical errors

FINAL C PROGRAMMING CHECKLIST

You should be able to write a C program that can:

- ☐ Display output
- ☐ Accept user input
- ☐ Declare variables
- ☐ Use different data types
- ☐ Perform calculations
- ☐ Use arithmetic operators
- ☐ Compare values
- ☐ Use logical operators
- ☐ Make decisions
- ☐ Use `switch`
- ☐ Use `for` loops
- ☐ Use `while` loops
- ☐ Use `do...while`
- ☐ Create functions
- ☐ Pass parameters
- ☐ Return values
- ☐ Use recursion
- ☐ Create arrays

- [ ] Process arrays using loops
- [ ] Work with strings
- [ ] Use string functions
- [ ] Create pointers
- [ ] Pass pointers to functions
- [ ] Create structures
- [ ] Create arrays of structures
- [ ] Read files
- [ ] Write files
- [ ] Append to files
- [ ] Build menu-driven programs
- [ ] Debug C programs
- [ ] Build a complete C project

QUICK REFERENCE: BASIC C PROGRAM

```
#include <stdio.h>

int main() {

    // Variables
    int number;

    // Input
    printf("Enter a number: ");
    scanf("%d", &number);

    // Processing
    if (number > 0) {
        printf("Positive number");
    } else if (number < 0) {
        printf("Negative number");
    } else {
        printf("Zero");
    }

    return 0;
}
```

This basic pattern contains the major stages of a structured program:

```
HEADER FILE
     ↓
MAIN FUNCTION
     ↓
VARIABLE DECLARATION
     ↓
INPUT
     ↓
PROCESSING
     ↓
OUTPUT
     ↓
END
```

END OF STRUCTURED PROGRAMMING IN C MODULE

Main skills gained: problem solving, algorithm design, C syntax, variables, operators, decision making, loops, functions, arrays, strings, pointers, structures, file handling, debugging, and development of complete menu-driven applications.